

BÁO CÁO ĐỒ ÁN MÔN HỌC

(Đồ án phát triển ứng dụng)

Lớp: IT003.Q21.CTTN

SINH VIÊN THỰC HIỆN

Mã sinh viên: 25521787

Họ và tên: Võ Quốc Thịnh

TÊN ĐỀ TÀI: Game Pacman

CÁC NỘI DUNG CẦN BÁO CÁO:

1. Giới thiệu đồ án:

a. Mô tả chung về ứng dụng:

Dự án là một phiên bản mô phỏng (clone) của tựa game arcade kinh điển Pac-Man. Người chơi điều khiển nhân vật chính để thu thập các hạt đậu trong mê cung.

b. Các CTDL và giải thuật đã được sử dụng: (cần làm rõ các câu hỏi: what, why)

- Mảng 2D (2D Array / Ma trận):

- What:** Cấu trúc dữ liệu dùng để lưu trữ và biểu diễn cấu trúc của bản đồ mê cung. Mỗi phần tử trong ma trận tương ứng với một trạng thái của ô
- Why:** Mảng 2D cho phép truy xuất tọa độ ngẫu nhiên trong thời gian $O(1)$. Điều này giúp thao tác kiểm tra va chạm (collision detection) diễn ra nhanh chóng, tối ưu hóa hiệu suất thay vì tính toán hình học phức tạp.

- Hàng đợi (Queue)

- What:** Cấu trúc dữ liệu hoạt động theo nguyên tắc **FIFO (First-In-First-Out)**, được khai báo trong

hàm `bfs_shortest_path()` tại `algorithms.py` dưới dạng `queue = collections.deque([start])`. Dùng để lưu trữ các đỉnh (tọa độ lưới) đang chờ được duyệt.

2. **Why:** Hàng đợi đảm bảo thuật toán BFS duyệt các ô theo từng cấp độ — tất cả các ô cách start đúng k bước được duyệt xong trước khi duyệt các ô cách k+1 bước. Thao tác `popleft()` và `append()` trên deque đều có thời gian $O(1)$, giúp BFS hoạt động hiệu quả

- **Giải thuật Tìm kiếm theo chiều rộng (Breadth-First Search - BFS):**

1. **What:** Thuật toán duyệt đồ thị lưới trong hàm `bfs_shortest_path(start, target, grid)` tại `algorithms.py`. Bắt đầu từ vị trí Ghost, BFS lần lượt: (1) lấy đỉnh đầu hàng đợi, (2) kiểm tra có phải đích không, (3) thêm tất cả đỉnh kề chưa duyệt vào hàng đợi, (4) lặp lại cho đến khi tìm thấy đích hoặc hết đỉnh.
2. **Why:** Trong mê cung Pac-Man, mọi bước di chuyển đều có chi phí bằng 1 (đồ thị không trọng số). BFS **đảm bảo** tìm được đường đi ngắn nhất, giúp Ghost luôn chọn hướng tối ưu nhất để tiếp cận Pac-Man.

- **Virtual Surface và Smooth Scaling**

1. **What:** Game được vẽ trước lên một `virtual_surface` có kích thước cố định theo cấu hình, sau đó dùng `pygame.transform.smoothscale()` để phóng to hoặc thu nhỏ vừa với màn hình thật.
2. **What:** Game được vẽ trước lên một `virtual_surface` có kích thước cố định theo cấu hình, sau đó dùng `pygame.transform.smoothscale()` để phóng to hoặc thu nhỏ vừa với màn hình thật.

- **Pygame Mixer Channel cho âm thanh nhiều kênh**

1. **What:** Sử dụng `pygame.mixer.set_num_channels(8)` và phân tách các kênh âm thanh như `eat_channel`, `siren_channel`, `ghost_channel`. Các file âm thanh được dùng gồm `eating.mp3`, `eat-pill.mp3`, `eat-ghost.mp3`, `siren.mp3`.
2. **Why:** Trong game Pac-Man, nhiều âm thanh có thể xảy ra gần như cùng lúc: tiếng ăn hạt, tiếng còi nền, tiếng ăn Energizer, tiếng ăn Ghost. Nếu chỉ dùng một kênh phát âm thanh, hiệu ứng mới có thể ghi đè hiệu ứng cũ. Việc tách kênh giúp âm thanh rõ ràng, tự nhiên và không bị ngắt đột ngột.

- **Socket, Thread và Pickle cho Multiplayer bước đầu**

1. **What:** Trong thư mục `Multiplayer_Python`, `server.py` dùng socket TCP, `threading` để xử lý nhiều client và `pickle` để đóng gói dữ liệu trạng thái game. `network.py` định nghĩa lớp `Network` cho client gửi input và nhận state từ server.
2. **Why:** Multiplayer cần một tiến trình trung tâm giữ trạng thái game thống nhất. Server chịu trách nhiệm cập nhật vị trí, điểm, Ghost, Pac-Man và gửi dữ liệu mới về các client. Cách này giúp tránh việc mỗi máy tự tính một trạng thái khác nhau.

2. Quá trình thực hiện

a. Tuần 1: Khởi tạo hệ thống, Mảng 2D và Cơ chế di chuyển :

- Thiết lập môi trường: Khởi tạo dự án với thư viện `pygame` và xây dựng vòng lặp trò chơi (Game Loop).
- Ứng dụng Mảng 2D biểu diễn bản đồ: Tải ma trận 2 chiều để lưu trữ cấu trúc bản đồ. Xây dựng hàm `render` để duyệt mảng và hiển thị các khối hình học lên màn hình.
- Xử lý Logic di chuyển & Va chạm $O(1)$: Nhận tín hiệu điều hướng từ người chơi. Tọa độ của Pac-Man được biểu diễn trực tiếp trên ma trận. Nếu giá trị tại ô đó là tường ('1'), nhân vật bị chặn lại.

- Tương tác cơ bản: Cập nhật giá trị ô ma trận từ hạt đậu ('0') thành đường trống (' ') khi nhân vật đi ngang qua.

b. Tuần 2: Xây dựng AI tìm đường (BFS) và script demo minh họa:

i. Cài đặt module `algorithms.py` :

1. `get_valid_neighbors(pos, grid, occupied_positions)` – Tìm tất cả ô kề đi được (không phải tường '1', không bị Ghost khác chiếm, có xử lý Tunnel Wrap Around khi $nc < 0$ hoặc $nc \geq \text{len}(\text{grid}[0])$).
2. `bfs_shortest_path(start, target, grid, occupied_positions)` – BFS tìm đường ngắn nhất, trả về bước đi tiếp theo.
3. `random_walk_algorithm(start, grid, occupied_positions)` – Chọn ngẫu nhiên ô kề hợp lệ, dùng làm fallback.

c. Tuần 3: Hoàn thiện Offline Mode và tích hợp khung chạy ứng dụng

- Hoàn thiện menu và vòng lặp game:

1. Xây dựng `main_menu()` để hiển thị màn hình chờ, chọn độ khó Easy/Hard và chọn số lượng Ghost.
2. Xây dựng `game_loop()` để xử lý input, cập nhật Pac-Man, cập nhật Ghost, kiểm tra va chạm, kiểm tra thắng/thua và render toàn bộ màn chơi
3. Bổ sung xử lý phím **ESC** để quay lại menu an toàn, đồng thời dừng âm thanh nền khi rời khỏi ván chơi.

- Tích hợp âm thanh:

1. Khởi tạo hệ thống âm thanh bằng `pygame.mixer.init()`.

2. Load các âm thanh chính: ăn hạt, ăn Energizer, ăn Ghost và siren nền.
 3. Tách kênh phát âm thanh để tránh việc các hiệu ứng ghi đè lẫn nhau.
- Tích hợp launcher trong app.py
 1. Tạo menu tổng cho ứng dụng với 3 lựa chọn: Offline Mode, Host Multiplayer Server và Join Multiplayer.
 2. Tái sử dụng game_loop() và main_menu() của chế độ offline trong app.py.
 3. Khi host multiplayer, chương trình tự chạy Multiplayer_Python/server.py, hiển thị IP máy chủ và cho phép người chơi tham gia vào phòng.
 4. Khi join multiplayer, người chơi nhập IP server và chọn vai trò Pac-Man hoặc Ghost.
 - Xây dựng Multiplayer bước đầu :
 1. server.py quản lý trạng thái tập trung của phòng chơi, danh sách Pac-Man, danh sách Ghost người chơi, AI Ghost và trạng thái bắt đầu game.
 2. client.py nhận state từ server và render bản đồ, Pac-Man, Ghost, điểm số, màn hình chờ.
 3. network.py đóng gói thao tác kết nối, gửi input và nhận dữ liệu phản hồi.
 4. Server dùng thread riêng cho từng client để nhiều người chơi có thể kết nối cùng lúc
2. Kết quả đạt được
 - Hoàn thiện được chế độ Offline Mode với menu, gameplay, âm thanh, đồ họa, hiệu ứng điểm số và điều kiện thắng/thua.
 - Ghost có thêm trạng thái Frightened, giúp gameplay giống Pac-Man gốc hơn và tạo thêm chiều sâu chiến thuật.

- Ứng dụng có launcher tổng trong app.py, hỗ trợ chọn Offline Mode hoặc chuyển sang luồng Multiplayer.
- Xây dựng được nền tảng Multiplayer ban đầu bằng socket TCP: server giữ trạng thái game, client gửi input và nhận state để render.
- Mã nguồn được tổ chức rõ hơn theo các module: main.py cho offline, app.py cho launcher, entities.py cho thực thể, algorithms.py cho thuật toán, Multiplayer_Python cho phần mạng. Xây dựng được nền tảng chuẩn bị cho việc tích hợp AI tìm đường vào tuần sau.

3. Tài liệu tham khảo

- Tài liệu Pygame Display và Surface: <https://www.pygame.org/docs/ref/display.html>
- Tài liệu Pygame Mixer: <https://www.pygame.org/docs/ref/mixer.html>
- Tài liệu Python Socket Programming: <https://docs.python.org/3/library/socket.html>
- Tài liệu Python Threading: <https://docs.python.org/3/library/threading.html>
- Giáo trình Cấu trúc dữ liệu và Giải thuật: nội dung về ma trận, hàng đợi, tập hợp, đồ thị và BFS.
- Tài liệu lập trình thư viện Pygame.

4. Phụ lục 1: Giới thiệu (demo) kết quả

<https://youtu.be/M3yMTWfRsUY>

5. Phụ lục 2: docstring

- Module Docstring:

```
"""
author : vqthinh
WEEK 1 DEMO: Basic 2D Arrays, Map Rendering, and Movement.
This script demonstrates the fundamental concept of treating
a 2D matrix
as a game world grid. It features a simplified Pacman with
basic wall
```

```
collision.  
"""
```

- Class BasicPacman:

```
"""  
  
author : vqthinh  
  
A simplified version of the Pacman entity for the initial  
week 1 demonstration.  
  
Focuses on discrete movement within a 2D grid matrix.  
  
"""
```

- Hàm update (Collision & Movement):

```
"""  
  
author : vqthinh  
  
Updates Pacman's position based on queued input and grid  
boundaries.  
  
Demonstrates O(1) wall collision by indexing directly into  
the 2D grid array.  
  
Args: grid (list[list[str]]): The grid matrix being  
navigated.  
  
"""
```

- Hàm draw:

```
"""  
  
author : vqthinh  
  
Draw a simple yellow circle to stands for Pacman  
  
"""
```

- Hàm draw_grid:

```
"""  
  
author : vqthinh  
  
Renders the game map from the 2D matrix.  
  
Args:  
  
surface (pygame.Surface): The Pygame display surface.  
  
grid (list[list[str]]): The 2D array representation of the  
map.  
  
"""
```

- Hàm bfs-shortest_path:

```
"""  
  
HARD DIFFICULTY AI (Breadth-First Search):  
Logic: Level-order traversal to find the shortest path in an  
unweighted graph.
```


Guarantees the absolute shortest path to the target (Pacman) by exploring all possible paths level-by-level using a Queue.

Data Structures:

- Queue (collections.deque): For $O(1)$ enqueue/dequeue operations.
- Dictionary (parent_map): To track visited nodes and reconstruct the path.

Args:

start (tuple): The starting (row, col) position.

target (tuple): The target (row, col) position (usually Pacman).

grid (list[list[str]]): The 2D grid matrix.

occupied_positions (set, optional): Positions of other ghosts.

Returns:

tuple: The immediate next (row, col) position on the shortest path.

Complexity:

Time: $O(V + E)$ where V is the number of grid cells and E is the number of valid moves.

Space: $O(V)$ to store the visited nodes in the parent_map and queue.

"""

- **Hàm random_walk_algorithm:**

```
"""
EASY DIFFICULTY AI / BFS FALLBACK:
Logic: Random selection of valid neighbors.

This algorithm picks a random valid direction at each step.
In demo_tuan2.py, it only activates as fallback when BFS
cannot find a path to the target (target fully enclosed).

Args:
    start (tuple): The current (row, col) position of the
ghost.
    grid (list[list[str]]): The 2D grid matrix.
    occupied_positions (set, optional): Positions of other
ghosts.

Returns:
    tuple: The next (row, col) position to move to.

Complexity:
    Time: O(1)
"""
```

- **Hàm get_valid_neighbors:**

```
"""
Finds all navigable adjacent cells (Up, Down, Left, Right)
from a given position.
```

```
A cell is considered navigable if it is within grid
boundaries, not a wall ('1'),
and not currently occupied by another entity. Includes
Tunnel Wrap Around logic.
```

Args:

```
    pos (tuple): The current (row, col) position.
    grid (list[list[str]]): The 2D grid matrix.
    occupied_positions (set, optional): Positions of other
ghosts to avoid collision.
```

Returns:

```
    list[tuple]: A list of navigable (row, col) neighbor
positions.
```

Complexity:

```
    Time: O(1) - Constant time as it only checks 4
directions.
```

```
"""
```

- Hàm `game_loop()` trong `main.py`:

```
"""
Runs the main offline Pac-Man gameplay loop.
Handles player input, entity updates, collision checks,
scoring,
audio events, visual effects, win/lose states and
rendering.
"""
```

- Hàm `build_game_state()` trong `Multiplayer_Python/server.py`:

```
"""
Builds the authoritative game state dictionary on the
server.
The state includes map data, Pac-Man players, Ghost
players,
AI ghosts, score, room status and sound events for
clients.
"""
```

- Hàm `frighten()` trong `entities.py`:

```
"""
Activates frightened mode for a ghost.
During this state, the ghost becomes vulnerable, moves
slower,
changes its visual appearance and can be eaten by Pac-Man.
"""
```

- Hàm `create_wall_surface()` trong `main.py`:

```
"""
Creates a static glowing surface for the map walls.
Draws wall connections with layered alpha lines to create
a neon glow effect while avoiding redrawing expensive wall
effects from scratch every frame.
"""
```